

TECHNICAL IMPLEMENTATION ROADMAP

Integrated Platform Roadmap

*From Phase 0 Data Pipeline to Production-Grade
Operational Intelligence Platform — A Step-by-Step
Technical Guide*

Timeline: 24 Weeks (6 Months) | **Phases:** 0 through 5

Version: 1.0 | **Date:** June 2026

Table of Contents

Introduction & Strategy

Phase 0: Current State Recap

Phase 1: Foundation Integration (Weeks 1-3)

1.1 Tasks & Implementation

1.2 Database Schema Design

1.3 Deliverables

Phase 2: Schema & API Layer (Weeks 3-6)

2.1 Dual-Schema Architecture

2.2 FastAPI Implementation

Phase 3: Platform UI (Weeks 6-10)

3.1 Next.js Frontend Architecture

3.2 Key Pages & Components

Phase 4: Advanced Intelligence (Weeks 10-16)

4.1 Enhanced Correlation Engine

4.2 AI-Assisted Root Cause Analysis

Phase 5: Production & Scale (Weeks 16-24)

5.1 Infrastructure & Deployment

5.2 Monitoring, Alerting & Auth

Summary & Milestones

Introduction & Strategy

This roadmap provides a detailed, step-by-step technical guide for transforming the current Phase 0 data pipeline into a production-grade operational intelligence platform. The strategy is built on a core insight from our architecture comparison: **the TATA Motors team has built the best-in-class data collection engine, and the Naxis team has built the right platform architecture.** The path forward is not to choose one over the other, but to integrate them into a single cohesive system.

The roadmap spans **24 weeks (6 months)**, organized into 6 phases from Phase 0 (current state) through Phase 5 (production deployment). Each phase builds incrementally on the previous one, ensuring that value is delivered at every milestone and that the architecture remains stable and testable throughout.

The guiding principles for this roadmap are: (1) **Preserve what works** — the BasePoller, retry logic, mock mode, and 12-field schema are production-ready and should be kept intact; (2) **Layer on capabilities** — each phase adds a new layer (persistence, API, UI, intelligence, production) without breaking existing functionality; (3) **Deliver value early** — by Phase 3 (week 10), the platform should have a working UI that network operators can actually use; (4) **Plan for scale** — every technical decision in early phases should not prevent horizontal scaling, multi-tenancy, or advanced analytics in later phases.

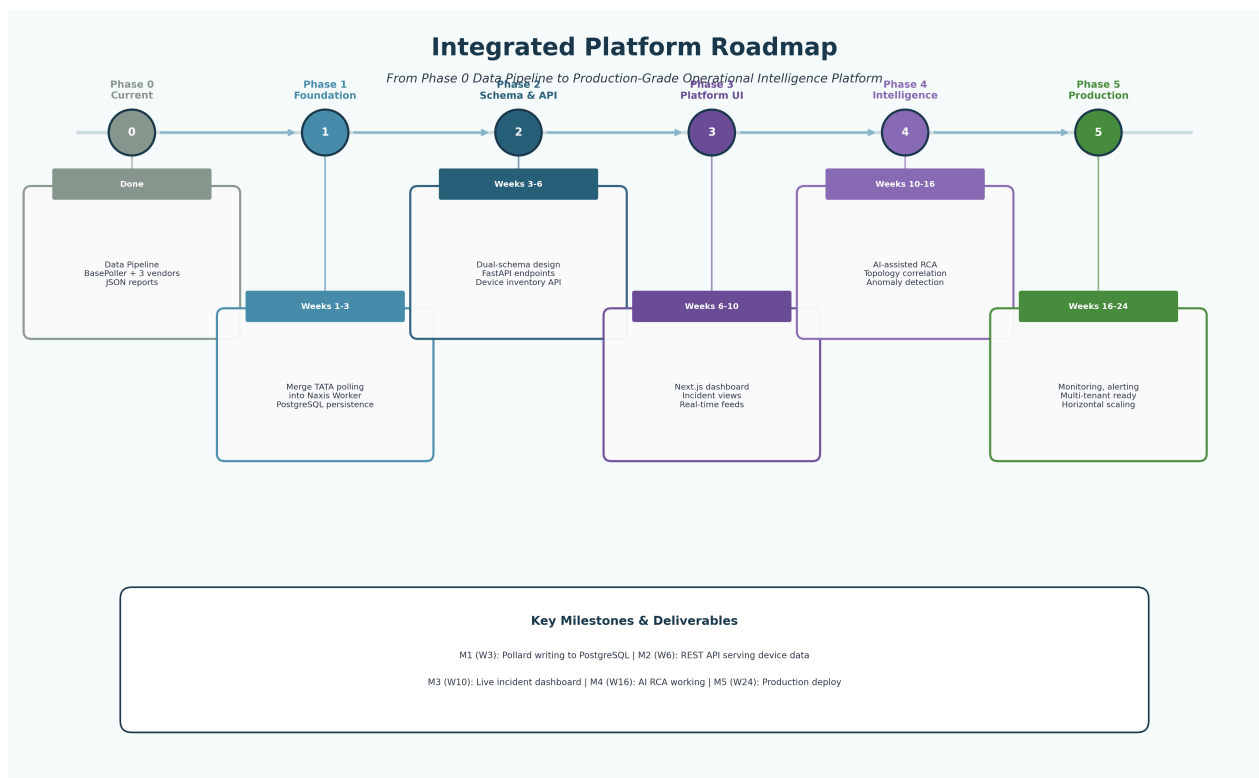


Figure 1: Integrated Platform Roadmap — 6 phases from current state to production

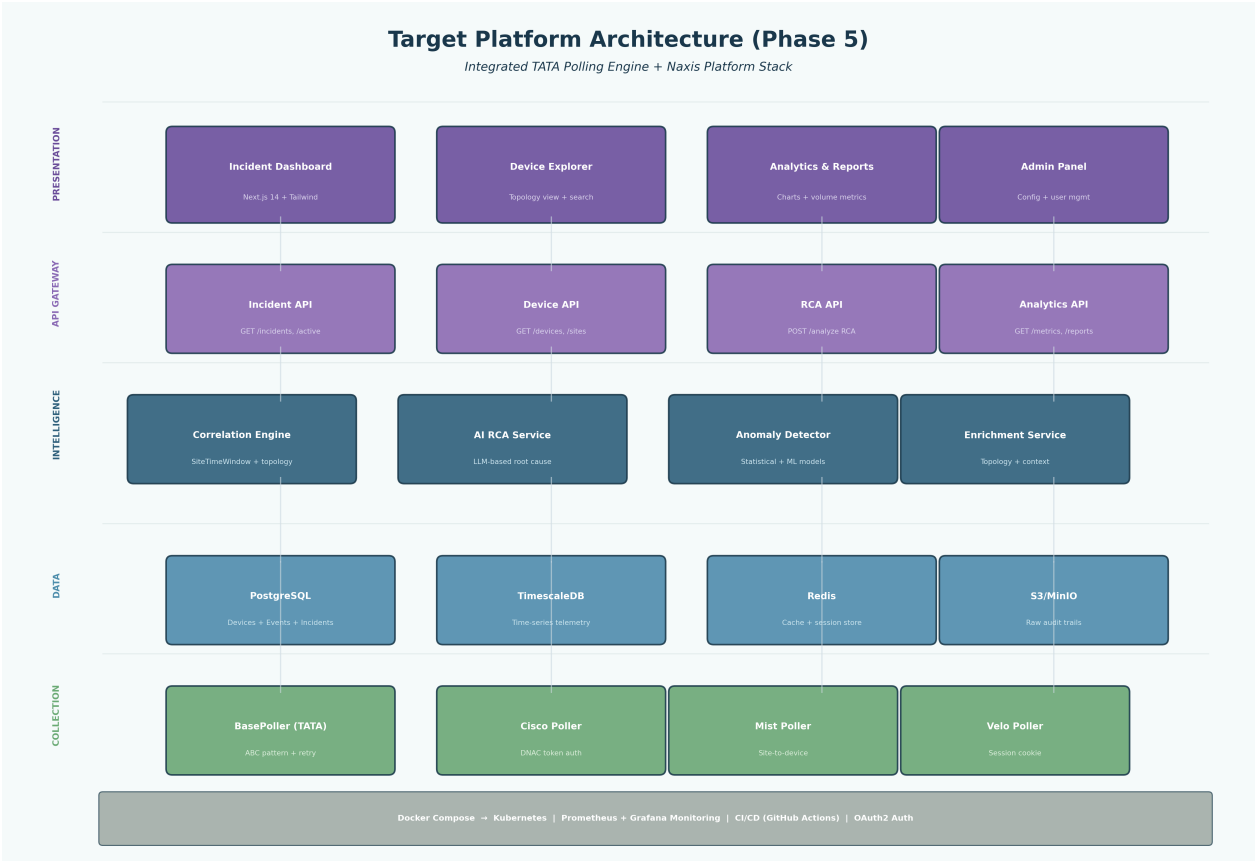


Figure 2: Target Platform Architecture (Phase 5) — The fully integrated system

Phase 0: Current State Recap

Before planning the next steps, let us establish a clear understanding of what exists today. Phase 0 has delivered a working data collection and normalization pipeline with the following components:

Component	Location	Status	Quality
BasePoller (ABC)	<code>pollers/base_poller.py</code>	Complete	Production-grade (retry, mock, session)
CiscoPoller	<code>pollers/cisco_poller.py</code>	Complete	Token auth, pagination
MistPoller	<code>pollers/mist_poller.py</code>	Complete	Site-to-device traversal
VeloPoller	<code>pollers/velo_poller.py</code>	Complete	Session cookie auth
CommonFieldsExtractor	<code>validators/common_fields.py</code>	Complete	12-field schema, MAC normalization
CrossPlatformCorrelator	<code>validators/cross_platform.py</code>	Complete	MAC/Serial matching
YAML Configs	<code>config/*.yaml</code>	Complete	Per-platform settings
pytest Tests	<code>tests/*.py</code>	Complete	Poller, extractor, correlator coverage
JSON Reports	<code>output/reports/*.json</code>	Complete	5 report types generated

The current architecture has no database, no REST API, no user interface, no reasoning engine beyond MAC matching, and no deployment automation. The output is a set of JSON files that must be read and interpreted manually. This is the starting point from which all subsequent phases will build.

Phase 1: Foundation Integration (Weeks 1-3)

Merge the TATA polling engine into a containerized backend with PostgreSQL persistence

1.1 Tasks & Implementation

Phase 1 is about creating the foundational infrastructure that everything else will build upon. The primary goal is to take the existing polling code and make it write to a PostgreSQL database instead of (or in addition to) JSON files. This requires three major workstreams: containerization, database setup, and pollard integration.

Step 1: Project Restructure

Create a new monorepo structure that will house the entire platform. This structure follows industry conventions and supports both the backend (Python) and frontend (TypeScript) codebases:

```

tata-network-platform/
├── backend/
│   ├── api/                # FastAPI service (Phase 2)
│   ├── worker/             # Polling + processing (Phase 1)
│   │   ├── collectors/     # TATA pollards moved here
│   │   │   ├── base.py     # BasePoller (from base_poller.py)
│   │   │   ├── cisco.py    # CiscoPoller
│   │   │   ├── mist.py     # MistPoller
│   │   │   └── velo.py     # VeloPoller
│   │   ├── processors/     # Normalization logic
│   │   │   └── normalizer.py # CommonFieldsExtractor
│   │   ├── config/         # YAML configs
│   │   ├── fixtures/       # Mock JSON fixtures
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   └── shared/             # Models shared between API and Worker
│       └── models.py        # SQLAlchemy models (Phase 1)
├── frontend/               # Next.js app (Phase 3)
├── database/
│   ├── migrations/         # Alembic migrations
│   └── init/
│       └── 001_schema.sql   # Initial schema
├── docker-compose.yml
├── Makefile
└── README.md

```

Step 2: Add PostgreSQL with SQLAlchemy

Install the required Python packages: `sqlalchemy`, `psycopg2-binary`, `alembic`, and `pydantic`. Create SQLAlchemy models that map the 12-field common schema to database tables. The key tables are `devices` (for normalized device inventory), `raw_responses` (for immutable audit trails), and `platforms` (for vendor configuration).

```

# backend/shared/models.py
from sqlalchemy import Column, String, DateTime, JSON, Integer, Float, create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.sql import func

Base = declarative_base()

class Device(Base):
    __tablename__ = "devices"

    id = Column(Integer, primary_key=True)
    platform = Column(String(20), nullable=False, index=True) # cisco, mist, velo
    device_id = Column(String(100), nullable=False)
    hostname = Column(String(255))
    ip_address = Column(String(45))
    mac_address = Column(String(17), index=True)
    serial_number = Column(String(100), index=True)
    platform_type = Column(String(100))
    software_version = Column(String(100))
    site = Column(String(255), index=True)
    uptime_seconds = Column(Integer)
    reachability = Column(String(20))
    last_seen = Column(DateTime(timezone=True))
    management_state = Column(String(50))
    raw_data = Column(JSON) # Full vendor response preserved
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), onupdate=func.now())

    __table_args__ = (
        # Ensure uniqueness per platform + device_id
        UniqueConstraint('platform', 'device_id', name='uix_device_platform'),
    )

class RawResponse(Base):
    __tablename__ = "raw_responses"

    id = Column(Integer, primary_key=True)
    platform = Column(String(20), nullable=False)
    endpoint = Column(String(500))
    response_data = Column(JSON, nullable=False)
    status_code = Column(Integer)
    collected_at = Column(DateTime(timezone=True), server_default=func.now())

    # Immutable audit trail – rows are never updated or deleted

```

Step 3: Modify BasePoller to Write to PostgreSQL

The critical change in Phase 1 is modifying the `_save_raw` method and adding a new `_persist_devices` method. The raw responses should continue to be saved as JSON files (for the immutable audit trail) but should also be inserted into the `raw_responses` table. The normalized devices should be upserted into the `devices` table using SQLAlchemy's `on_conflict_do_update` equivalent.


```

# Add to BasePoller.__init__
from shared.models import Base, Device, RawResponse
from sqlalchemy.orm import sessionmaker

# Database connection from environment
self.db_engine = create_engine(os.getenv("DATABASE_URL", "postgresql:// ..."))
Base.metadata.create_all(self.db_engine)
self.Session = sessionmaker(bind=self.db_engine)

def _persist_devices(self, devices: List[Dict[str, Any]]) → int:
    """Upsert normalized devices into PostgreSQL."""
    session = self.Session()
    count = 0
    try:
        for device_dict in devices:
            device = Device(
                platform=self.platform,
                device_id=device_dict.get("device_id"),
                hostname=device_dict.get("hostname"),
                ip_address=device_dict.get("ip_address"),
                mac_address=device_dict.get("mac_address"),
                serial_number=device_dict.get("serial_number"),
                platform_type=device_dict.get("platform_type"),
                software_version=device_dict.get("software_version"),
                site=device_dict.get("site"),
                uptime_seconds=device_dict.get("uptime_seconds"),
                reachability=device_dict.get("reachability"),
                last_seen=device_dict.get("last_seen"),
                management_state=device_dict.get("management_state"),
                raw_data=device_dict.get("_raw"),
            )
            session.merge(device) # Upsert on conflict
            count += 1
        session.commit()
        logger.info("Persisted %d %s devices to PostgreSQL", count, self.platform)
    except Exception as exc:
        session.rollback()
        logger.error("Failed to persist devices: %s", exc)
        raise
    finally:
        session.close()
    return count

```

Step 4: Docker Compose Setup

Create a `docker-compose.yml` that runs PostgreSQL and the worker service. This provides a reproducible development environment that any team member can start with a single command.

```

# docker-compose.yml
version: "3.9"
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: naxis
      POSTGRES_PASSWORD: naxis_dev
      POSTGRES_DB: network_platform
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./database/init:/docker-entrypoint-initdb.d
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U naxis"]
      interval: 5s
      timeout: 5s
      retries: 5

  worker:
    build: ./backend/worker
    environment:
      DATABASE_URL: postgresql://naxis:naxis_dev@postgres:5432/network_platform
      # Vendor credentials from .env
    volumes:
      - ./backend/worker/fixtures:/app/fixtures
      - ./backend/worker/config:/app/config
    depends_on:
      postgres:
        condition: service_healthy
    command: python -m run_poll # New entry point

volumes:
  postgres_data:

```

1.2 Database Schema Design

The initial PostgreSQL schema must support three core use cases: (1) storing normalized device inventory with fast lookup by MAC, serial, and site; (2) preserving immutable raw API responses for audit purposes; and (3) providing a foundation for the event and incident tables that will be added in Phase 2. The schema below achieves these goals with appropriate indexes and constraints.

Table	Purpose	Key Columns	Indexes
devices	Normalized device inventory	platform, device_id, mac_address, serial_number, site	mac_address, serial_number, site, platform+device_id (unique)
raw_responses	Immutable audit trail	platform, endpoint, response_data, collected_at	collected_at, platform
platforms	Vendor configuration	name, config_yaml, enabled	name (primary key)
polling_runs	Track each pollard execution	started_at, completed_at, status, records_collected	started_at, status

1.3 Deliverables & Milestones

Milestone M1 (End of Week 3): Foundation Complete

- All three pollards (Cisco, Mist, Velo) running inside Docker and writing normalized data to PostgreSQL
- Mock mode works end-to-end: `docker-compose up` populates the database with fixture data
- Raw responses preserved in both JSON files (audit trail) and PostgreSQL (queryable)
- Alembic migrations set up for schema versioning
- pytest test suite updated to test database persistence layer

Phase 2: Schema & API Layer (Weeks 3-6)

Build the dual-schema architecture and FastAPI REST endpoints

2.1 Dual-Schema Architecture

Phase 2 introduces the concept of a **dual-schema architecture** that supports both device inventory (from the TATA Motors 12-field schema) and event-driven telemetry (from the Naxis UnifiedEvent schema). This is essential because a complete operational platform needs both: device inventory provides the topology context for understanding where events occur, and event streams provide the operational signals that drive incident detection.

The device schema (already defined in Phase 1) is inventory-centric — it answers "what devices exist and what are their static properties?" The event schema is new in this phase and answers "what is happening in the network right now?" Events are time-series data: alarms, state changes, metric thresholds exceeded, authentication failures, and so on.

```
# backend/shared/models.py – Event schema additions

class NetworkEvent(Base):
    __tablename__ = "network_events"

    id = Column(Integer, primary_key=True)
    event_type = Column(String(50), nullable=False, index=True)
    # Foreign key to devices table for topology context
    device_id = Column(Integer, ForeignKey("devices.id"), nullable=True)
    platform = Column(String(20), nullable=False, index=True)
    severity = Column(String(20), default="info") # critical, warning, info
    message = Column(Text)
    event_data = Column(JSON) # Vendor-specific event payload
    source_ip = Column(String(45))
    site = Column(String(255), index=True)
    timestamp = Column(DateTime(timezone=True), nullable=False, index=True)
    correlated = Column(Boolean, default=False) # Has this been processed?
    created_at = Column(DateTime(timezone=True), server_default=func.now())

    # Relationship
    device = relationship("Device", back_populates="events")

    __table_args__ = (
        Index('ix_events_site_time', 'site', 'timestamp'),
        Index('ix_events_platform_time', 'platform', 'timestamp'),
    )

# Add to Device class:
events = relationship("NetworkEvent", back_populates="device")
```

The `correlated` flag is a simple but important mechanism: when the correlation engine (Phase 4) processes an event, it sets this flag to `True`. This prevents duplicate processing and allows the system to track which events have been incorporated into incidents. The `device_id` foreign key links events to the device inventory, enabling topology-aware correlation — if an event occurs at Site A, the system can quickly find all devices at Site A and assess the blast radius.

2.2 FastAPI Implementation

With data now persisted in PostgreSQL, the next step is to expose it through a REST API. FastAPI is the ideal choice because it provides automatic OpenAPI documentation, Pydantic-based request/response validation, and excellent async support. The API service lives in `backend/api/` and runs as a separate Docker container.

```

# backend/api/main.py
from fastapi import FastAPI, Depends, Query
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from shared.models import Device, NetworkEvent, get_db
from shared.schemas import DeviceResponse, DeviceList, EventList

app = FastAPI(
    title="TATA Network Platform API",
    version="1.0.0",
    docs_url="/docs", # Auto-generated Swagger UI
)

# CORS for frontend (Phase 3)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"], # Next.js dev server
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/devices", response_model=DeviceList)
def list_devices(
    platform: str | None = Query(None, description="Filter by vendor platform"),
    site: str | None = Query(None, description="Filter by site/location"),
    limit: int = Query(100, ge=1, le=1000),
    offset: int = Query(0, ge=0),
    db: Session = Depends(get_db),
):
    """List all normalized network devices with optional filtering."""
    query = db.query(Device)
    if platform:
        query = query.filter(Device.platform == platform)
    if site:
        query = query.filter(Device.site.ilike(f"%{site}%"))

    total = query.count()
    devices = query.offset(offset).limit(limit).all()
    return {"total": total, "items": devices, "limit": limit, "offset": offset}

@app.get("/devices/{device_id}", response_model=DeviceResponse)
def get_device(device_id: int, db: Session = Depends(get_db)):
    """Get detailed information for a specific device."""
    device = db.query(Device).filter(Device.id == device_id).first()
    if not device:
        raise HTTPException(status_code=404, detail="Device not found")
    return device

```

These endpoints provide the data foundation for the frontend that will be built in Phase 3. The `/devices` endpoint supports pagination and filtering by platform and site — exactly what the incident dashboard needs to display device counts by location. The `/events` endpoint supports time-range queries, which the correlation engine will use to find events within a specific window.

Milestone M2 (End of Week 6): API Layer Complete

- FastAPI service running with auto-generated Swagger docs at `/docs`
- CRUD endpoints for devices and events with filtering and pagination
- Pydantic v2 schemas for request validation and response serialization
- CORS configured for Next.js frontend integration
- API tests with pytest and TestClient

Phase 3: Platform UI (Weeks 6-10)

Build the Next.js frontend with incident dashboards and device exploration

3.1 Next.js Frontend Architecture

Phase 3 is where the platform becomes visible and usable. The frontend is built with Next.js 14 (App Router), TypeScript, Tailwind CSS, and shadcn/ui — the same stack used by the Naxis platform and widely adopted in the industry for operational dashboards. The frontend communicates with the FastAPI backend via the REST endpoints built in Phase 2.

The project structure for the frontend follows Next.js 14 conventions:

```

frontend/
├── app/                                # Next.js App Router
│   ├── layout.tsx                     # Root layout with providers
│   ├── page.tsx                       # Dashboard (incident list)
│   ├── devices/
│   │   └── page.tsx                   # Device explorer
│   ├── incidents/
│   │   └── [id]/
│   │       └── page.tsx               # Incident detail page
│   └── analytics/
│       └── page.tsx                   # Charts and reports
├── components/
│   ├── ui/                            # shadcn/ui components
│   ├── IncidentCard.tsx               # Incident summary card
│   ├── DeviceTable.tsx                # Sortable/filterable device table
│   ├── EventTimeline.tsx              # Visual event timeline
│   └── BlastRadiusMap.tsx              # Topology visualization
├── lib/
│   ├── api.ts                         # Typed API client (fetch wrapper)
│   └── utils.ts                       # Helper functions
├── hooks/
│   └── useDevices.ts                  # TanStack Query hooks
├── types/
│   └── index.ts                       # TypeScript interfaces matching Pydantic schemas
├── public/
├── tailwind.config.ts
├── next.config.mjs                    # API proxy to backend
└── package.json

```

3.2 Key Pages & Components

The three most important pages are the Incident Dashboard (main landing page), the Device Explorer (for browsing the network inventory), and the Incident Detail Page (for investigating specific issues). Each page is designed to provide network operators with the information they need to understand network health and respond to incidents.

The **Incident Dashboard** (/) displays a list of active incidents with real-time auto-refresh every 10 seconds. It uses TanStack Query's `useQuery` hook with a `refetchInterval` of 10000ms to achieve this. Each incident is rendered as an `IncidentCard` component showing severity (color-coded), affected site, device count, time since creation, and a brief summary.

The **Device Explorer** (/devices) provides a searchable, filterable, and sortable table of all devices across all platforms. It supports filtering by platform (Cisco, Mist, Velo), site, reachability status, and software version. This page directly consumes the `/devices` API endpoint with query parameters for filtering.

The **Incident Detail Page** (`/incidents/[id]`) is the most complex page. It displays: (1) a narrative summary of the incident, (2) the computed blast radius (affected sites and devices), (3) a timeline of correlated events, (4) a probable cause section (populated by the AI RCA service in Phase 4), and (5) action buttons for acknowledging, escalating, or resolving the incident.

```
# frontend/lib/api.ts – Typed API client
const API_BASE = process.env.NEXT_PUBLIC_API_URL || "http://localhost:8000";

export async function fetchDevices(params?: DeviceFilterParams): Promise {
  const query = new URLSearchParams();
  if (params?.platform) query.set("platform", params.platform);
  if (params?.site) query.set("site", params.site);
  query.set("limit", String(params?.limit || 100));

  const res = await fetch(`${API_BASE}/devices?${query}`);
  if (!res.ok) throw new Error(`API error: ${res.status}`);
  return res.json();
}

export async function fetchEvents(params?: EventFilterParams): Promise {
  const query = new URLSearchParams();
  if (params?.site) query.set("site", params.site);
  if (params?.severity) query.set("severity", params.severity);

  const res = await fetch(`${API_BASE}/events?${query}`);
  if (!res.ok) throw new Error(`API error: ${res.status}`);
  return res.json();
}
```

The `next.config.mjs` file should be configured to proxy API requests during development, avoiding CORS issues:

```
// frontend/next.config.mjs
/** @type {import('next').NextConfig} */
const nextConfig = {
  async rewrites() {
    return [
      {
        source: '/api/:path*',
        destination: 'http://localhost:8000/:path*',
      },
    ];
  },
};
export default nextConfig;
```


Milestone M3 (End of Week 10): Live Dashboard

- Incident dashboard with real-time auto-refresh (10-second polling)
- Device explorer with search, filter, and sort capabilities
- Incident detail page with event timeline and blast radius
- Responsive design working on desktop and tablet
- End-to-end test: mock data flows from pollard → database → API → UI

Phase 4: Advanced Intelligence (Weeks 10-16)

Build the correlation engine, AI-assisted RCA, and anomaly detection

4.1 Enhanced Correlation Engine

Phase 4 is where the platform transitions from a data display tool to an operational intelligence system. The correlation engine analyzes events, groups related ones into incidents, and provides the reasoning layer that makes the platform truly valuable to network operators.

The engine should implement multiple correlation strategies that can be combined:

Rule Type	Trigger	Example
SiteTimeWindow	Multiple events at same site within N minutes	3+ events at "Plant A" within 5 min → Incident
TopologyCascade	Downstream devices affected by upstream failure	Core switch down → all connected APs affected
MACCrossPlatform	Same device reporting issues on multiple platforms	Switch X shows alarm on both Cisco DNAC and Mist
PatternMatch	Recurring event pattern over time	Daily BGP flap at 2 AM → scheduled maintenance conflict

```

# backend/worker/correlation/engine.py
from datetime import datetime, timedelta
from typing import List, Dict
from sqlalchemy.orm import Session
from shared.models import NetworkEvent, Incident

class CorrelationEngine:
    def __init__(self, db: Session):
        self.db = db
        self.rules = [
            SiteTimeWindowRule(window_minutes=5, threshold=3),
            TopologyCascadeRule(),
            MACCrossPlatformRule(),
        ]

    def process_unprocessed_events(self) → List[Incident]:
        """Find all uncorrelated events and run them through rules."""
        events = self.db.query(NetworkEvent).filter(
            NetworkEvent.correlated == False
        ).order_by(NetworkEvent.timestamp).all()

        incidents = []
        for rule in self.rules:
            new_incidents = rule.evaluate(events, self.db)
            incidents.extend(new_incidents)

        # Mark processed events
        for event in events:
            event.correlated = True
        self.db.commit()

        return incidents

class SiteTimeWindowRule:
    def __init__(self, window_minutes: int = 5, threshold: int = 3):
        self.window = timedelta(minutes=window_minutes)
        self.threshold = threshold

    def evaluate(self, events: List[NetworkEvent], db: Session) → List[Incident]:
        # Group events by site
        by_site: Dict[str, List[NetworkEvent]] = {}
        for event in events:
            if event.site not in by_site:
                by_site[event.site] = []
            by_site[event.site].append(event)

        incidents = []
        for site, site_events in by_site.items():

```

4.2 AI-Assisted Root Cause Analysis

The AI RCA service uses a Large Language Model (LLM) to analyze incidents and suggest probable causes. This is not meant to replace human judgment, but to accelerate the investigation process by providing a starting hypothesis that the network operator can validate.

```

# backend/worker/intelligence/rca_service.py
import openai
from typing import List, Dict

class RCAService:
    def __init__(self, api_key: str):
        self.client = openai.AsyncOpenAI(api_key=api_key)

    async def analyze(self, incident: Incident, events: List[NetworkEvent]) → Dict:
        """Generate a root cause analysis for an incident."""
        # Build context from incident data
        context = self._build_context(incident, events)

        response = await self.client.chat.completions.create(
            model="gpt-4-turbo-preview",
            messages=[
                {
                    "role": "system",
                    "content": (
                        "You are a network operations expert. Analyze the following "
                        "network incident and provide: (1) a probable root cause, "
                        "(2) confidence level (high/medium/low), (3) recommended "
                        "actions, (4) blast radius assessment. Be concise."
                    ),
                },
                {"role": "user", "content": context},
            ],
            temperature=0.3, # Lower temperature for more deterministic output
            max_tokens=500,
        )

        analysis = response.choices[0].message.content
        return {
            "incident_id": incident.id,
            "analysis": analysis,
            "model": "gpt-4-turbo-preview",
            "generated_at": datetime.now(timezone.utc).isoformat(),
        }

    def _build_context(self, incident: Incident, events: List[NetworkEvent]) → str:
        lines = [
            f"Incident: {incident.title}",
            f"Site: {incident.site}",
            f"Severity: {incident.severity}",
            f"Device count: {incident.device_count}",
            f"Events ({len(events)}):",
        ]
        for event in events[:10]: # Limit context window

```

The RCA service is exposed through the API as a POST endpoint: `POST /incidents/{id}/analyze`. When called, it triggers the LLM analysis asynchronously (using Celery, introduced in this phase) and stores the result in the incident record. The frontend displays the analysis on the incident detail page with a confidence badge and a "Regenerate" button if the operator disagrees with the initial assessment.

Milestone M4 (End of Week 16): Intelligence Layer Complete

- Correlation engine with SiteTimeWindow, TopologyCascade, and MACCrossPlatform rules
- AI RCA service integrated with OpenAI API, generating structured analysis
- Anomaly detection using statistical baselines (z-score for metric deviations)
- Celery workers for async processing of correlation and RCA tasks
- WebSocket support for real-time incident notifications to the frontend

Phase 5: Production & Scale (Weeks 16-24)

Production hardening, monitoring, authentication, and horizontal scaling

5.1 Infrastructure & Deployment

Phase 5 transforms the development environment into a production-ready system. The Docker Compose setup used in development is replaced or augmented with Kubernetes for orchestration, which provides auto-scaling, rolling deployments, health checks, and service discovery.

The infrastructure stack includes:

Component	Development	Production	Purpose
Orchestration	Docker Compose	Kubernetes (EKS/AKS)	Container orchestration and scaling
Database	PostgreSQL container	Managed PostgreSQL (RDS/Cloud SQL)	Persistent storage with backups
Cache	Redis container	Managed Redis (ElastiCache)	Session store and API caching
Object Storage	Local volume	S3 / MinIO	Raw audit trail archives
CI/CD	Manual	GitHub Actions	Automated build, test, deploy
Secret Management	.env files	Azure Key Vault / AWS Secrets	Credential management

5.2 Monitoring, Alerting & Authentication

Production readiness requires three additional systems: monitoring (to observe platform health), alerting (to notify when things go wrong), and authentication (to control access).

Monitoring is implemented with Prometheus for metrics collection and Grafana for visualization. Key metrics include: API response times (p50, p95, p99), database query latency, worker queue depth (Celery), polling success/failure rates per vendor, incident correlation accuracy, and frontend page load times. These metrics are exposed through Prometheus endpoints on each service.

Alerting uses Prometheus Alertmanager to send notifications (Slack, PagerDuty, email) when thresholds are breached. Critical alerts include: polling failure rate above 10%, database connection pool exhausted, API error rate above 1%, and worker queue depth growing without bound.

Authentication uses Keycloak (or Azure AD B2C) for OAuth2/OIDC-based authentication. This provides SSO integration with TATA Motors' existing identity infrastructure, role-based access control (RBAC) for different operator levels (viewer, operator, admin), and audit logging of all user actions.

```
# backend/api/auth.py – OAuth2 integration
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer
import jwt

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

async def get_current_user(token: str = Depends(oauth2_scheme)) → User:
    """Validate JWT token from Keycloak and return user."""
    try:
        payload = jwt.decode(
            token,
            key=KEYCLOAK_PUBLIC_KEY,
            algorithms=["RS256"],
            audience="network-platform",
        )
        return User(
            id=payload["sub"],
            email=payload["email"],
            name=payload["name"],
            roles=payload.get("realm_access", {}).get("roles", []),
        )
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=401, detail="Invalid token")

# Protect API endpoints
@app.get("/admin/reports")
def admin_reports(user: User = Depends(get_current_user)):
    if "admin" not in user.roles:
        raise HTTPException(status_code=403, detail="Admin access required")
    return generate_admin_report()
```

Milestone M5 (End of Week 24): Production Deployed

- Platform running on Kubernetes with auto-scaling
- Prometheus + Grafana monitoring dashboards operational
- Alertmanager sending critical alerts to Slack/PagerDuty
- OAuth2 authentication with RBAC (viewer, operator, admin roles)
- CI/CD pipeline: commit → test → build → deploy in under 15 minutes
- Disaster recovery: database backups, documented runbooks, rollback procedures

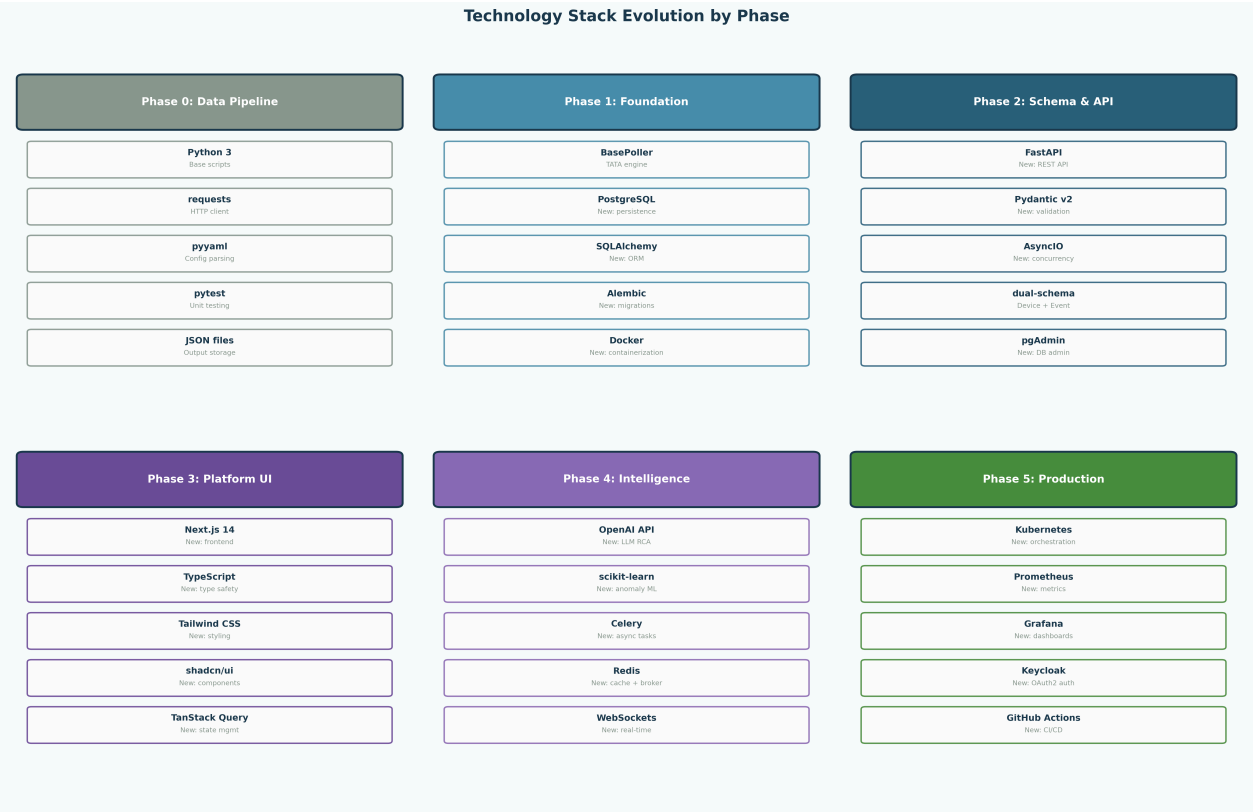


Figure 3: Technology Stack Evolution by Phase — What gets added at each stage

Summary & Key Milestones

Phase	Duration	Key Deliverable	Milestone
Phase 0	Complete	Data pipeline with 3 vendor pollards	JSON reports generated
Phase 1	Weeks 1-3	Pollards writing to PostgreSQL	M1: Docker + DB persistence working
Phase 2	Weeks 3-6	FastAPI with device/event endpoints	M2: REST API serving data
Phase 3	Weeks 6-10	Next.js incident dashboard	M3: Live UI with real-time refresh
Phase 4	Weeks 10-16	Correlation engine + AI RCA	M4: Automated incident detection
Phase 5	Weeks 16-24	Kubernetes + monitoring + auth	M5: Production deployment

Final Recommendation: Start with Phase 1 immediately. The work done in Phase 0 is not wasted — it is the engine that powers everything. Each phase builds incrementally, and by Milestone M3 (week 10), the platform will have a working user interface that demonstrates real value to stakeholders. This phased approach minimizes risk, allows for course correction at every milestone, and ensures that technical debt is addressed before it becomes unmanageable.

Integrated Platform Roadmap

From Phase 0 to Production in 24 Weeks

TATA Motors Network Resilience Platform

June 2026

